Final Project Prediction

Owen Ling

- UW ID: 20904591

Important: follow the structure of the document and **summarize important handlings in the summary section**. The structure here is kept at a minimum, feel free if you have more subsections and details to add. Delete this paragraph in the end.

Summary

Preprocessing

Transformation (if any)

- salary: I log-transformed the response variate salary.
- The following predictor variables were log-transformed: pts, goal, age, takeaway, giveaway, pim, and hits.
- date: converted to numerical variable using as.numeric(as.date()) function.
- the following variables were converted into numerical factors: nat, team, pos.
- The following variables were converted to 2nd-degree polynomials: gp and pm.

New Variables

- pts_toi: interaction between points and time on ice.
- goal_toi: intercation between goal and time on ice.
- toi_gp: interaction between games played and time on ice.
- pim_hits: interaction between penalty minutes and hits.
- pm_toi: interaction between plus-minus and time on ice.
- take_toi: interaction between takeaway and time on ice.
- give_toi: interaction between giveaway and time on ice.
- take_pm: interactio between taekaway and plus_minus.
- toi_inj: interaction between time on ice and games injured.
- toi_age: interaction between tim eon ice and age.

- injure_pts: interaction between games injured and points.

Model

Summarize your model here: could be the formula for a smoothing method, random forest, etc.

My model is a random forest model with the following formula: "log(salary)~.", which is all the default variables(some transformed as mentioned above in the "transformation section") and the interaction variables in the "new variables" section.

Model Building

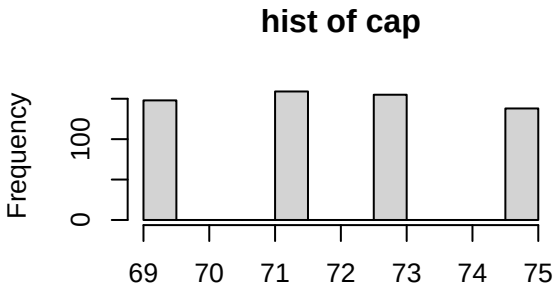
Summarize your model building process, such as model selection strategy for a smoothing method, or parameter combinations searched for a random forest. Report your minimized error measure. If you have attempted multiple methods, report the minimized error measure for each.

1.Preprocessing

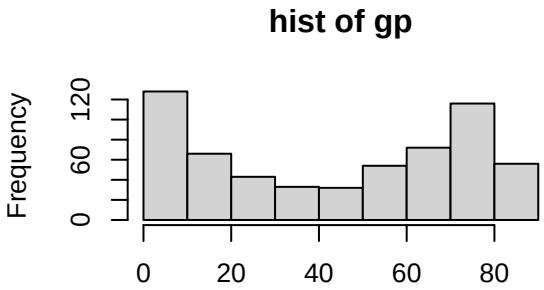
1.1 Loading data

```
load("final.Rdata")
```

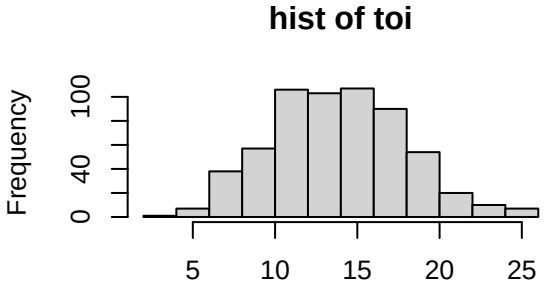
```
library(moments)
#check the skewness of the distribution for each variable to determine whether log transformation is ne
num_vars = c("cap", "gp", "toi", "pts", "goal", "age",
             "takeaway", "giveaway", "relCorsi", "pim", "hits", "pm")
par(mfrow = c(2, 2))
for (c in num_vars) {
  hist(dtrain[[c]], main = paste("hist of", c), xlab = paste("skewness:", skewness(dtrain[[c]])))
}
```



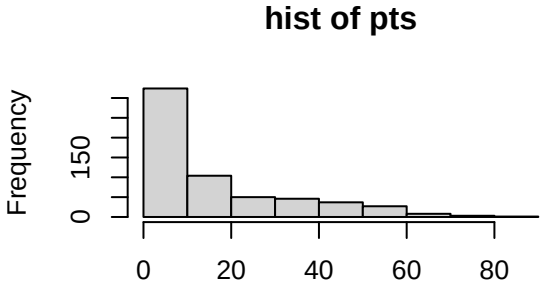
skewness: −0.0936026569967484



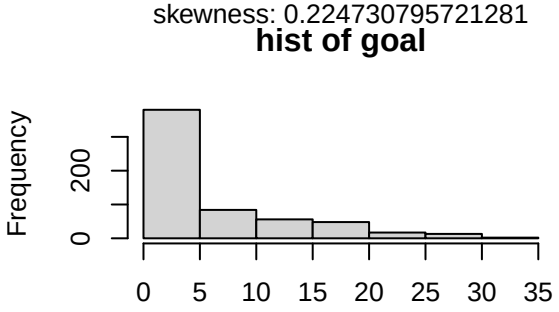
skewness: −0.144244300812307



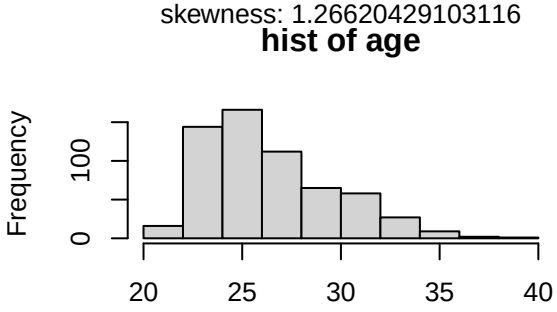
skewness: 0.224730795721281



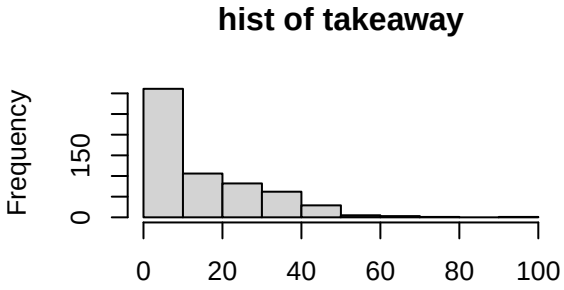
skewness: 1.26620429103116



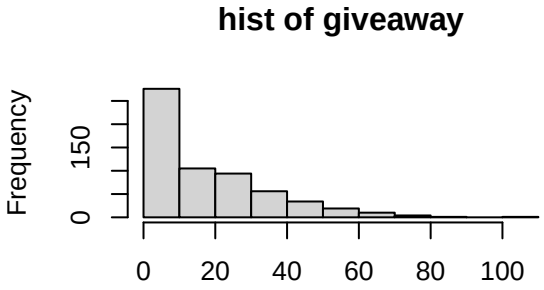
skewness: 1.42047184586093



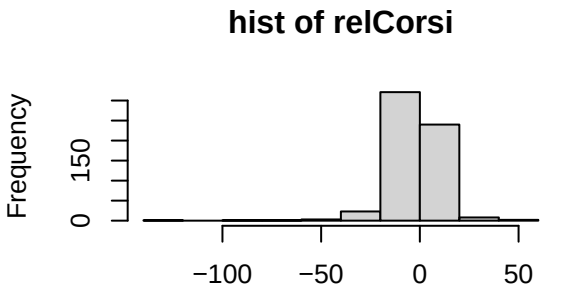
skewness: 0.816621417987994



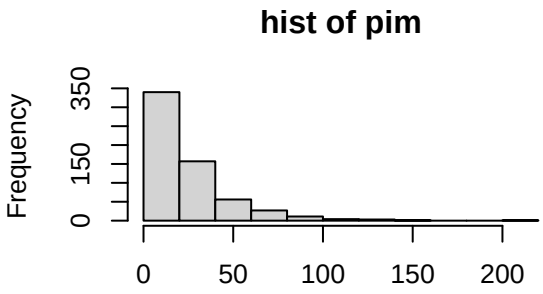
skewness: 1.19949410194912



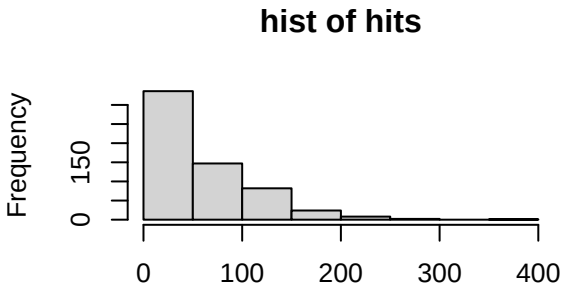
skewness: 1.22681808693223



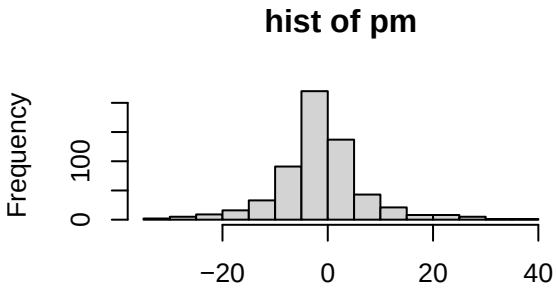
skewness: −2.84004354505318



skewness: 2.14662204925463



skewness: 1.43361567352069



skewness: 0.274946050879413

```
set.seed(444)
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
library(ranger)
```

```
##
## Attaching package: 'ranger'
```

```
## The following object is masked from 'package:randomForest':
##
##     importance
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
##
## Attaching package: 'ggplot2'
```

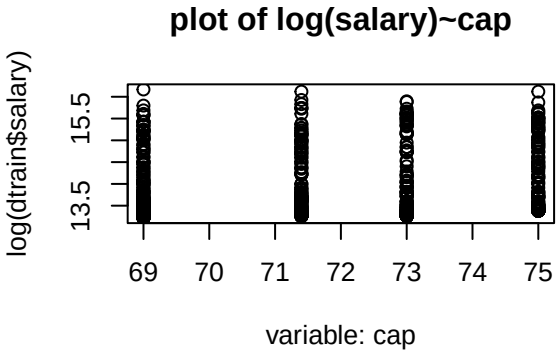
```
## The following object is masked from 'package:randomForest':
##
##     margin
```

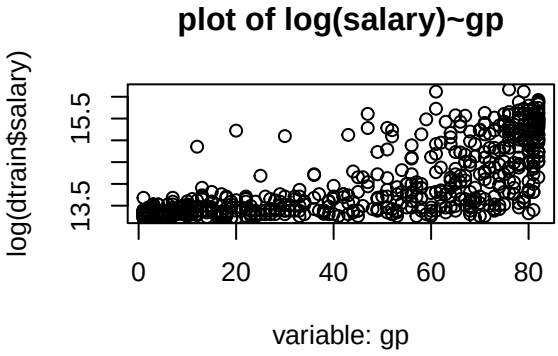
```
## Loading required package: lattice
```

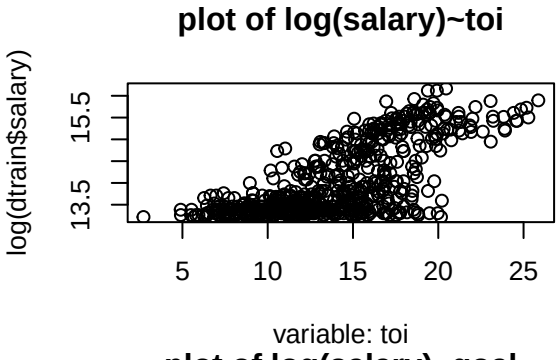
```
  dtrain$date = as.numeric(as.Date(dtrain$date))

  #skewness:
  dtrain$pts = log(dtrain$pts+1)
  dtrain$goal = log(dtrain$goal+1)
```

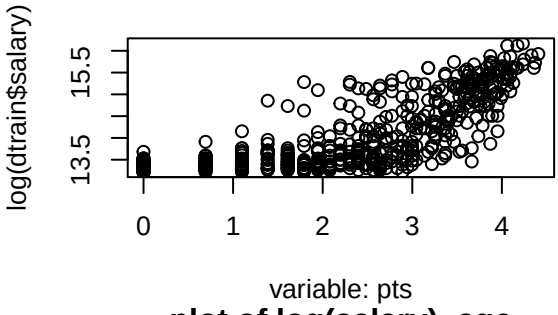
```r
  dtrain$age = log(dtrain$age)
  dtrain$takeaway = log(dtrain$takeaway+1)
  dtrain$giveaway = log(dtrain$giveaway+1)
  dtrain$pim = log(dtrain$pim+1)
  dtrain$hits = log(dtrain$hits+1)

  #categorical->numbers
  dtrain$nat = as.integer(factor(dtrain$nat))
  dtrain$team = as.integer(factor(dtrain$team))
  dtrain$pos = as.integer(factor(dtrain$pos))

  #interactions
  dtrain$pts_toi= dtrain$pts*dtrain$toi
  dtrain$goal_toi = dtrain$goal*dtrain$toi
  dtrain$toi_gp = dtrain$toi*dtrain$gp
  dtrain$pim_hits =  dtrain$pim*dtrain$hits
  dtrain$pm_toi = dtrain$pm*dtrain$toi

  dtrain$take_toi = dtrain$takeaway*dtrain$toi
  dtrain$give_toi = dtrain$giveaway*dtrain$toi
  dtrain$take_pm = dtrain$takeaway*dtrain$pm
  dtrain$toi_inj= dtrain$toi*dtrain$injure
  dtrain$toi_age = dtrain$toi*dtrain$age


  dtrain$injure_pts=dtrain$injure*dtrain$pts
```

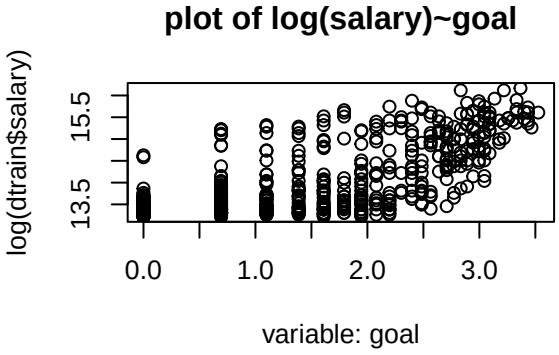
```r
#plot each variable against salary to examine each individual relationship in detail,
#in order to determine whether polynomial/exponential transformations are needed.
par(mfrow = c(2, 2))
for (c in num_vars) {
  plot(log(dtrain$salary)~dtrain[[c]], main = paste0("plot of log(salary)~", c)
       , xlab = paste("variable:", c), data = dtrain)
}
```

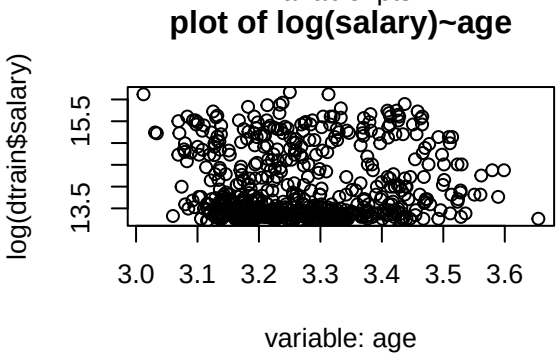

plot of log(salary)~cap
log(dtrain$salary)
13.5
15.5
69
70
71
72
73
74
75
variable: cap

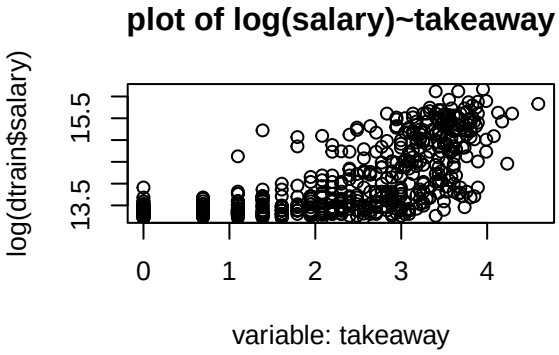

plot of log(salary)~gp
log(dtrain$salary)
13.5
15.5
0
20
40
60
80
variable: gp

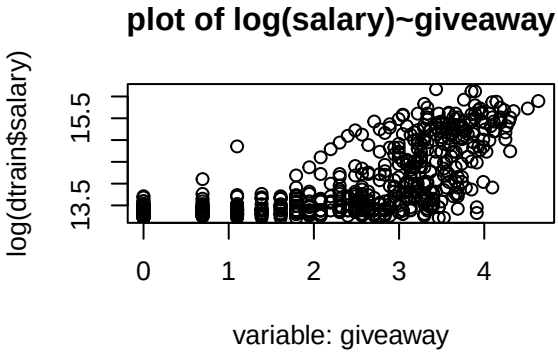

plot of log(salary)~toi
log(dtrain$salary)
13.5
15.5
5
10
15
20
25
variable: toi

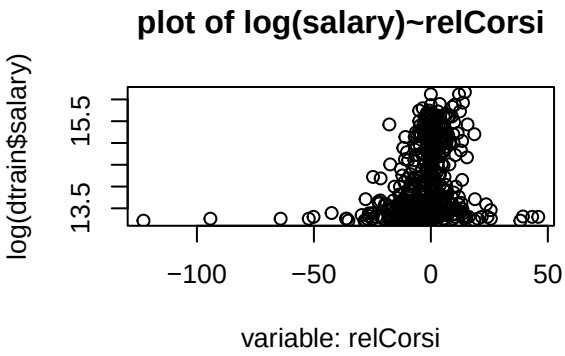

plot of log(salary)~pts
log(dtrain$salary)
13.5
15.5
0
1
2
3
4
variable: pts

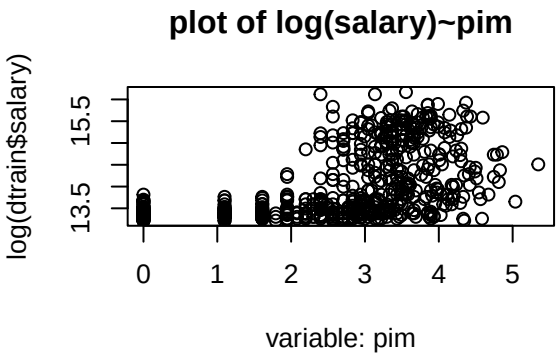

plot of log(salary)~goal
log(dtrain$salary)
13.5
15.5
0.0
1.0
2.0
3.0
variable: goal

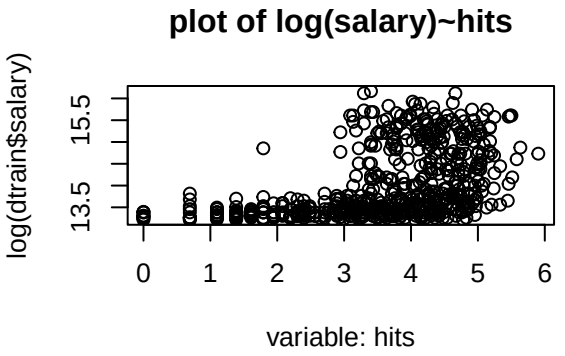

plot of log(salary)~age
log(dtrain$salary)
13.5
15.5
3.0
3.1
3.2
3.3
3.4
3.5
3.6
variable: age

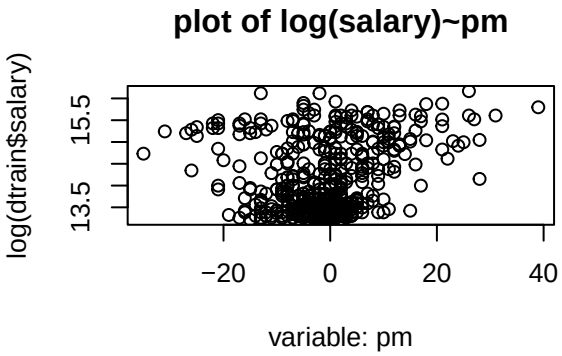

plot of log(salary)~takeaway
log(dtrain$salary)
13.5
15.5
0
1
2
3
4
variable: takeaway


plot of log(salary)~giveaway
log(dtrain$salary)
13.5
15.5
0
1
2
3
4
variable: giveaway










```
#polynomials
  dtrain$gp = poly(dtrain$gp, degree=2)
  dtrain$pm = poly(dtrain$pm, degree= 2)
```

2.Model Building

```
set.seed(444)
train_control <- trainControl(method = "cv", number = 5, search = "grid")
  fit_test <- train(
    log(salary) ~ .,
    data = dtrain,
    method = "ranger", #this is another function for random forest.
    trControl = train_control,
    tuneLength = 50,
    verbose=1,
    importance = "permutation"
  )
```

```
## note: only 29 unique complexity parameters in default grid. Truncating the grid to 29 .
```

```
print(fit_test)
```

```
## Random Forest
##
## 600 samples
##  28 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
```

```
## Summary of sample sizes: 480, 480, 480, 480, 480
## Resampling results across tuning parameters:
##
##   mtry  splitrule   RMSE       Rsquared   MAE
##    2    variance    0.3381855  0.8174844  0.2347837
##    2    extratrees  0.3441674  0.8171668  0.2428565
##    3    variance    0.3357889  0.8175264  0.2307781
##    3    extratrees  0.3384075  0.8206032  0.2374104
##    4    variance    0.3353273  0.8171915  0.2294680
##    4    extratrees  0.3364391  0.8205967  0.2339410
##    5    variance    0.3326749  0.8192180  0.2273901
##    5    extratrees  0.3347431  0.8208371  0.2314928
##    6    variance    0.3327828  0.8187219  0.2272422
##    6    extratrees  0.3337178  0.8217397  0.2303668
##    7    variance    0.3326754  0.8180337  0.2276965
##    7    extratrees  0.3331650  0.8223034  0.2305140
##    8    variance    0.3329351  0.8180919  0.2266294
##    8    extratrees  0.3334628  0.8214144  0.2301429
##    9    variance    0.3343718  0.8159155  0.2279918
##    9    extratrees  0.3313289  0.8228644  0.2286853
##   10    variance    0.3331852  0.8169350  0.2276046
##   10    extratrees  0.3330471  0.8214136  0.2296245
##   11    variance    0.3337937  0.8162335  0.2276451
##   11    extratrees  0.3315178  0.8227383  0.2276272
##   12    variance    0.3336841  0.8159066  0.2276902
##   12    extratrees  0.3311808  0.8224738  0.2282329
##   13    variance    0.3316542  0.8181267  0.2275124
##   13    extratrees  0.3322744  0.8210155  0.2282624
##   14    variance    0.3351988  0.8142329  0.2289555
##   14    extratrees  0.3323936  0.8209807  0.2284052
##   15    variance    0.3342872  0.8145897  0.2279617
##   15    extratrees  0.3304843  0.8228255  0.2279023
##   16    variance    0.3351300  0.8140374  0.2288996
##   16    extratrees  0.3308859  0.8222443  0.2279692
##   17    variance    0.3352718  0.8137037  0.2293882
##   17    extratrees  0.3301917  0.8228166  0.2273833
##   18    variance    0.3350299  0.8138521  0.2285326
##   18    extratrees  0.3305816  0.8221652  0.2270742
##   19    variance    0.3374432  0.8110189  0.2306547
##   19    extratrees  0.3300398  0.8228163  0.2271097
##   20    variance    0.3361149  0.8122170  0.2296014
##   20    extratrees  0.3307090  0.8224582  0.2276443
##   21    variance    0.3361456  0.8120758  0.2291121
##   21    extratrees  0.3293512  0.8230537  0.2257187
##   22    variance    0.3383039  0.8099839  0.2314901
##   22    extratrees  0.3309084  0.8214336  0.2273507
##   23    variance    0.3366822  0.8115241  0.2295143
##   23    extratrees  0.3326703  0.8199261  0.2287589
##   24    variance    0.3375051  0.8104319  0.2315543
##   24    extratrees  0.3302516  0.8221924  0.2274317
##   25    variance    0.3377611  0.8104025  0.2311105
##   25    extratrees  0.3302366  0.8224385  0.2271748
##   26    variance    0.3392267  0.8086700  0.2320247
##   26    extratrees  0.3307242  0.8216225  0.2273304
```

```
##   27    variance    0.3404485  0.8068576  0.2327574
##   27    extratrees  0.3308628  0.8211415  0.2274015
##   28    variance    0.3410142  0.8071719  0.2329306
##   28    extratrees  0.3318432  0.8202905  0.2280332
##   29    variance    0.3407764  0.8065498  0.2338033
##   29    extratrees  0.3331771  0.8187621  0.2292392
##   30    variance    0.3417826  0.8061001  0.2337436
##   30    extratrees  0.3300863  0.8224466  0.2269035
##
## Tuning parameter 'min.node.size' was held constant at a value of 5
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were mtry = 21, splitrule = extratrees
##  and min.node.size = 5.
```

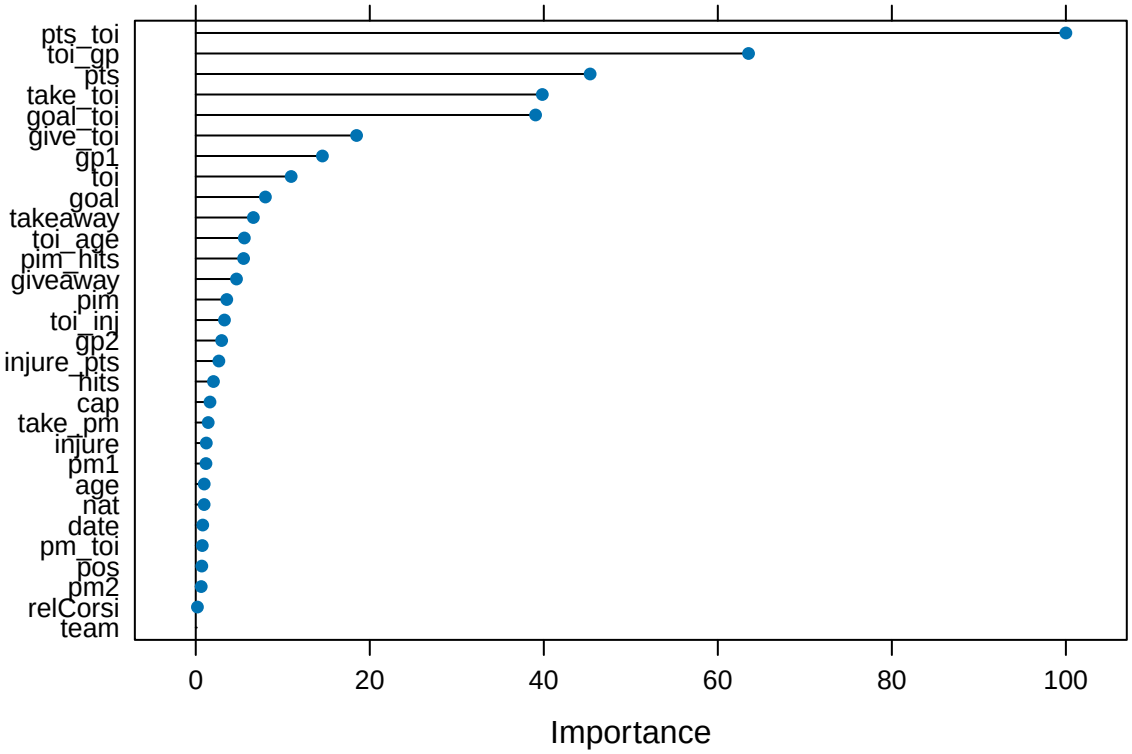
```
fit_test$bestTune
```

```
##    mtry  splitrule min.node.size
## 40   21 extratrees             5
```

```
imp = varImp(fit_test)
print(imp)
```

```
## ranger variable importance
##
##   only 20 most important variables shown (out of 30)
##
##            Overall
## pts_toi    100.000
## toi_gp      63.529
## pts         45.326
## take_toi    39.831
## goal_toi    39.055
## give_toi    18.483
## gp1         14.557
## toi         10.967
## goal         8.005
## takeaway     6.619
## toi_age      5.584
## pim_hits     5.497
## giveaway     4.684
## pim          3.565
## toi_inj      3.300
## gp2          2.976
## injure_pts   2.656
## hits         2.036
## cap          1.637
## take_pm      1.431
```

```
plot(varImp(fit_test), top = 30)
```



I saw that pts, goal, and toi had the most impact on improving the accuracy of the model, so I created many interactions involving these 3 variables that I found to be meaningful.

3.Evaluation

```
rm(list=ls())
# first set working directory to the data location
load("final.Rdata")
# final.Rdata contains dtrain and dtest

# load user-defined FinalModel function
source("20904591.R")
# call the function
tmp <- system.time(res <- FinalModel(dtrain, dtest))
```

```
## note: only 29 unique complexity parameters in default grid. Truncating the grid to 29 .
```

```
# time used, this is the "Running_time" you report in your R source file
tmp[3]
```

```
## elapsed
##  62.133
```

```
dim(res)
```

```
## [1] 400   2
```

```
# should be 400 by 2

head(res)
```

```
##   Id    salary
## 1  1  704343.7
## 2  2 1054452.8
## 3  3  636236.0
```

```
## 4  4 3534313.6
## 5  5 5961097.6
## 6  6  908943.8
```